CarbonTrack

Backend context & remaining work

For the team building the Java backend. Status date: 5 Nov 2026. Only the React frontend prototype exists today; everything on the backend is still to be built.

1. What we are building

An enterprise Java platform where Indian companies log activity data (electricity, fuel, travel, logistics). It converts that data to CO_2e, tracks it against reduction targets, checks eligibility for green incentives, and uses AI to forecast emissions, flag anomalies and suggest reductions. Standard: GHG Protocol (Scope 1, 2, 3) with BRSR-style reports for India.

Core calculation: **activity quantity × emission factor = CO_2e**, tagged with a scope (1, 2 or 3). Everything else (targets, incentives, dashboards, AI) is built on the stored EmissionResult rows.

2. Tech stack and architecture

Java 21 · Spring Boot 3 · Spring Security + JWT · Spring Data JPA · PostgreSQL · Flyway · Apache POI · JasperReports/OpenPDF · Spring AI · Tribuo · Docker Compose · GitHub Actions · AWS EC2 · Swagger · JUnit 5, Mockito, Testcontainers. Frontend: React (Vite) in `frontend/`.

Architecture: modular monolith. One deployable, one package per module. Modules talk through service interfaces, not each other's repositories, so a module can be split out later.

```
com.carbontrack: auth/ company/ factor/ ingestion/ calculation/ target/ incentive/ report/ ai/ alert/ common/ (config, exceptions, dto, audit)
```

3. Modules and responsibilities

Module	Responsibility	Key notes
auth + company	Companies, facilities, users, roles (Super Admin, Company Admin, Analyst, Auditor), JWT login	Multi-tenant: every row carries company_id; enforce in a repository filter, not only in controllers. Auditor is read-only.
factor	Emission factors versioned by source, region, year; Excel importer	Sources: CEA CO2 Baseline DB v20 (India grid, Scope 2), UK DESNZ (fuels, travel, freight, waste), US EPA Hub (backup), Climatiq API (live lookup, free tier 250 calls/month).
ingestion	Manual entry, CSV/Excel upload (POI), REST API, simulated IoT meter (scheduled job)	Validate unit vs category; reject duplicates for the same facility/category/period; every record stores its source channel.
calculation	Computes EmissionResult from ActivityRecord	Strategy pattern per scope/activity type. Store the factor id used, so results stay reproducible after factor updates.
target	Baseline year, reduction target, logged reduction actions (solar, EVs)	Progress = (baseline - current) / baseline, compared with a linear pace to the target year.
incentive	DB-configurable rules (e.g. more than 15% below baseline = Tier A) and simulated credits	Rules live in IncentiveRule rows, not code. IncentiveAward records each award.
report	Dashboard aggregates, BRSR-style PDF, Excel exports	Aggregation queries by month, scope and facility. PDF via JasperReports or OpenPDF.
ai	Forecasting and anomaly detection (Tribuo); LLM sustainability assistant (Spring AI)	Isolation-style anomaly score per facility/category/month; forecast with prediction band. The assistant should only see the caller's tenant data.

Module	Responsibility	Key notes
alert + audit	Spike and target-slip notifications; change history via Hibernate Envers	Alerts raised by a scheduled job after ingestion; audit log is append-only.

4. Core entities

Company, Facility, User, Role, EmissionFactor(source, category, unit, value, region, year), **ActivityRecord**(facility, category, quantity, unit, period), **EmissionResult**(activity, factor, co2e, scope), **Target, ReductionAction, IncentiveRule, IncentiveAward, Alert, AuditLog**.

Suggested relationships: Company 1-N Facility; Facility 1-N ActivityRecord; ActivityRecord 1-1 EmissionResult (N-1 EmissionFactor); Company 1-N Target / ReductionAction / IncentiveAward / Alert.

5. Data strategy

- **Emission factors are real data** from the sources listed above. Version every import (source + year), never overwrite in place.
- **Company data is hybrid.** Pick one real listed company (e.g. Infosys or Tata Steel) and take 3 years of Scope 1 and 2 totals from its BRSR (NSE filings or the company ESG page). Generate mock monthly, facility-wise activity data (Datafaker, seasonality, noise, injected anomalies) scaled to match those totals.
- Add 4-5 fully fake companies for multi-tenancy demos.
- **Demo claim:** "our engine reproduces the company's disclosed emissions within X%." X must come from an actual run, not be asserted.

6. Frontend status

A clickable prototype lives in `frontend/` (React 18 + Vite; run `npm install` then `npm run dev`, port 5173). It has 8 screens: Dashboard, Activity data, Emission factors, Targets & actions, Incentives, Reports, AI insights, Alerts & audit.

All numbers on screen are placeholders in `frontend/src/data.js`: company names, emissions, the grid factor (0.82 kg/kWh), forecast and chatbot replies are invented to show the layout. Nothing is calculated, and no API calls are made yet. Do not quote any of these figures; the real grid factor must be taken from CEA v20.

7. Proposed API contract (derived from the screens; to be agreed)

None of this is implemented. It is the minimum the UI needs, so backend and frontend can agree early.

Area	Endpoints (proposed)	Used by
Auth	POST /api/v1/auth/login; GET /api/v1/me	Sidebar user and role, route guards
Dashboard	GET /dashboard/summary (totals, vs baseline, intensity, health score); GET /dashboard/monthly?by=scope; GET /dashboard/facilities	KPI cards, stacked chart, scope donut, facility table
Activity	POST /activity; POST /activity/upload (CSV/xlsx); GET /activity?period=; POST /activity/preview (returns co2e without saving)	Entry form with live CO2e preview, bulk upload, records table
Factors	GET /factors?q=; POST /factors/import (xlsx); GET /factors/lookup (Climatiq)	Factor library
Targets	GET/PUT /targets; GET/POST /actions	Progress ring, action table
Incentives	GET /incentives/rules; GET /incentives/status; POST /incentives/evaluate	Tier cards and standing
Reports	POST /reports (type, period, format); GET /reports; GET /reports/{id}/download	Report builder and list
AI	GET /ai/forecast; GET /ai/anomalies; GET /ai/suggestions; POST /ai/chat	Forecast chart, anomaly table, chat
Alerts / audit	GET /alerts; PATCH /alerts/{id}; GET /audit	Alerts and audit log

All endpoints are tenant-scoped from the JWT. Document them in Swagger/OpenAPI so the frontend can generate types.

8. What is left to do

Status key: **Done** = exists today; **To do** = not started.

Area	Work remaining	Status
Backend skeleton	Spring Boot 3 / Java 21 project, package layout, Flyway baseline, Docker Compose with PostgreSQL, Swagger	To do
Auth & tenancy	Entities, JWT login, role-based access, tenant filter, seed users	To do
Emission factors	Entity + versioning, Excel importer (POI), load CEA v20 / DESNZ / EPA values, Climatiq client with call counter	To do
Ingestion	Manual + CSV/Excel + REST ingestion, validation, duplicate handling, IoT simulator scheduled job	To do
Calculation engine	Strategy per scope/activity, EmissionResult persistence, recalculation when a factor changes, unit tests with hand-checked cases	To do
Targets & actions	Baseline/target model, progress and pace calculation, action CRUD	To do
Incentive engine	Rule evaluation from DB, awards, simulated credits	To do
Dashboard & reports	Aggregation queries, BRSR-style PDF (OpenPDF/Jasper), Excel export	To do

Area	Work remaining	Status
AI	Tribuo anomaly detection and forecast; Spring AI assistant with tenant-scoped context	To do
Audit & alerts	Envers setup, alert generation job (spike, target slip), notifications	To do
Seed / demo data	Pick company, extract 3 yrs BRSR Scope 1+2, Datafaker generator scaled to totals, 4-5 fake tenants, inject anomalies, compute and report the reproduction error	To do
Testing	JUnit 5 + Mockito unit tests, Testcontainers integration tests, calculation accuracy tests	To do
DevOps	Dockerfile, GitHub Actions (build + test), AWS EC2 deploy, secrets handling	To do
Frontend UI	8 screens with mock data, design system, responsive layout	Done (prototype)
Frontend integration	Replace data.js with API client, login screen, loading/error states, role-based hiding, charts fed by real data, real file upload and PDF download	To do

9. Suggested build order

- **1.** Skeleton + auth/tenancy + Postgres/Flyway (unblocks everyone).
- **2.** Factor import and the calculation engine with tests. This is the heart of the product; get it right before building on it.
- **3.** Ingestion (manual + CSV first), then dashboard aggregates. Wire the frontend Dashboard and Activity pages to the real API.
- **4.** Seed-data generator and the "within X%" validation against the BRSR totals.
- **5.** Targets, incentives, alerts/audit.
- **6.** Reports (PDF), then AI (forecast, anomalies, assistant).
- **7.** CI/CD and EC2 deployment; demo rehearsal.

10. Risks and open questions

- **Factor mismatch.** DESNZ factors are UK/global and CEA covers only the grid. State the source and year on every result; do not present them as Indian-specific where they are not.
- **Reproduction accuracy.** The "within X%" claim depends on how cleanly BRSR totals can be split into facility activity data. Decide the acceptable X with the team before the demo.
- **Scope 3 is partial.** Only travel, logistics and waste are modelled; say so in reports.
- **Climatiq free tier is 250 calls/month.** Cache lookups in the factor table.
- **AI claims.** Forecast and anomaly outputs must come from the trained models on the generated data. The assistant's answers must be grounded in the tenant's own results, not generic advice.
- **Credits are simulated.** Label incentive awards as simulated everywhere in the UI and reports.
- **Open:** which listed company to use; React vs Thymeleaf is settled for now (React); who owns each module.

Sources: CEA CO2 Baseline Database User Guide v20 (cea.nic.in); UK Government GHG Conversion Factors for Company Reporting (gov.uk); US EPA GHG Emission Factors Hub; Climatiq docs (climatiq.io/docs).